

基于 Agent 的智能运维系统构建与研究

李盛鹏

学号：2351136

计算机科学与技术学院，数据科学与大数据技术

摘要—微服务系统的运行状态同时体现在资源指标、服务指标、数据库连接和应用日志等多源信号中。传统人工运维需要在多个面板和日志文件之间切换，容易出现告警割裂、重复上报和根因定位缓慢等问题。本文面向课程实验构建一个可复现的本地 AIOps Agent，使用 Docker 部署 Demo 微服务与 MySQL，使用 Python/FastAPI 编排本机采集、规则检测、根因分析、历史存储和网页展示，并在 WSL2 中运行 FreeAIOps 管理异常事件。系统先将主机、服务和日志转换为统一观测快照，再由 rule-v1 完成故障检测；异常观测通过兼容事件信封写入并从 FreeAIOps 取回，随后结合 Log 与 Metric 生成根因候选和需要人工确认的排查建议。配置 OpenAI-compatible/DeepSeek 后，LLM 只负责自然语言诊断增强，不参与采集和修复。20 个离线案例全部判定正确，准确率为 100%；异常检测平均耗时 0.004 ms，规则根因分析平均耗时 0.166 ms；最新回归测试共 50 项全部通过。实验表明，该方案在教学环境中具备较好的可解释性、可降级性和安全边界，但仍受固定阈值、案例规模和模型服务稳定性的限制。

Index Terms—AIOps, Agent, FreeAIOps, 故障检测, 根因分析, Docker, 大语言模型, 可观测性

I. 引言

A. 研究背景

随着服务拆分和容器化部署，单个故障可能同时表现为资源使用率升高、请求延迟增加、错误日志增多和数据库连接池紧张。若只观察单一指标，运维人员很难区分资源瓶颈、下游依赖故障和偶发噪声。AIOps 的基本思想是将监控、事件、知识和自动化能力结合起来，以较低人工成本完成异常发现、定位和处置 [1], [6]。

本课程项目关注一个具体问题：在不修改生产服务、不执行危险修复命令的约束下，如何把 Metrics 与 Logs 组织成可解释的诊断闭环。项目选择轻量级 Demo 微服务作为被运维对象，利用 Docker、MySQL、WSL2 和 Python Agent 构造跨运行时实验环境。该环境能够复现健康检查、指标采集、错误注入、事件持久化、根

因分析和网页展示，又避免复杂 Kubernetes 集群带来的额外变量。

B. 研究问题与贡献

本文回答三个问题：如何设计共享于离线评估、在线接口和网页展示的标准观测格式；如何在规则检测保持可解释的前提下，将异常证据转化为 Log+Metric 根因候选并接入 FreeAIOps；如何建立可降级和可审计的安全边界，避免模型输出执行未经批准的系统修改。主要贡献包括：

- 1) 构建采集、校验、检测、分析、事件管理、历史存储和可视化闭环；
- 2) 实现故障检测与 Log+Metric 根因分析两个职责独立的场景；
- 3) 设计 FreeAIOps 健康适配与兼容事件信封，支持 published/retrieved/deduplicated；
- 4) 使用 20 个案例、50 项自动化测试和真实本地链路验证系统表现。

C. 报告组织

第二节介绍相关工作和需求；第三节说明整体方案与选型；第四节说明环境搭建和关键配置；第五节介绍数据格式、采集与存储；第六节分析 Agent 设计实现；第七节介绍 API 与控制台；第八节给出测试评估和讨论；第九节描述典型故障演示；第十节复盘困难、解决过程和局限；最后给出结论、参考文献和附录。

II. 相关工作与需求分析

A. AIOps 与日志异常检测

Kephart 和 Chess 将自主计算描述为系统根据环境变化进行自我配置、修复和优化的愿景 [1]。Xu 等人研究了通过控制台日志挖掘检测大规模系统问题的方法 [2]；DeepLog 将日志模板序列转化为可预测序列 [3]；

表 I
系统功能需求与验收目标

场景	输入	输出与目标
故障检测	指标快照、可选日志	标签、严重程度、置信度和证据
根因分析	检测结果、Log、Metric	根因候选、证据引用和建议
本机采集	psutil、Demo、固定日志	标准观测, 缺失指标为 null
事件管理	异常观测	published、retrieved、deduplicated
可视化	API 结果、历史记录	监控、历史、问答和诊断书

LogAnomaly 融合日志序列与数量特征 [4]; He 等人讨论了日志解析对挖掘任务的影响 [5]。相关工作表明, 日志与指标联合分析比孤立阈值更有潜力, 但需要稳定输入格式和可追溯证据链。

本文先建立 rule-v1 规则基线, 而不是直接训练深度模型, 原因是课程实验更需要稳定、透明和可复核的输出; 规则可以说明具体指标、阈值和日志原因; 规则路径在 FreeAiOps 或 LLM 不可用时仍能运行。模型被定位为可选解释器, 而非唯一判定器。

B. 功能需求

C. 非功能需求与安全边界

系统应满足可解释、可复现、可降级和可测试。规则输出需保留阈值和日志证据; Docker Compose、脚本、案例和测试命令保证复现; FreeAiOps 离线或模型失败时 T4/T5 仍返回合法 JSON; 测试覆盖输入校验、采集、检测、分析、存储、适配和前端语法。采集器只读项目日志; Key 只在后端读取; 浏览器不接触 Key; requires_approval 固定为 true; auto_remediation_allowed 固定为 false; 不删除文件、不重启服务、不修改生产环境。

III. 整体技术方案与选型理由

A. 总体方案

系统采用“被运维对象-采集层-判断层-分析层-事件与存储-服务与展示”的分层方案。Docker Demo 提供 /health、/metrics 和 /error; 采集层使用 psutil、HTTP 和固定日志路径; 判断层先做格式校验, 再运行 rule-v1; 异常观测才进入 FreeAiOps 事件链路; 分析层结合 Log 与 Metric 生成规则根因或可选 LLM 诊断; SQLite 保存项目历史, FastAPI 向网页提供同源接口。

表 II
技术栈、职责与选择原因

层次	选型	职责与理由
被运维对象	Docker Demo	可控提供健康、指标和错误注入
Agent/API	Python、FastAPI、Uvicorn	组合模块并便于测试
采集	psutil、HTTP、固定日志	覆盖主机、应用和日志信号
检测/分析	rule-v1 与规则回退	稳定、低延迟、可解释
事件框架	FreeAiOps + MySQL	事件接收、持久化和查询
项目存储	SQLite	与框架数据隔离
模型	OpenAI-compatible/DeepSeek	可选自然语言诊断
前端	原生 HTML/CSS/JavaScript	本地同源控制台

```

flowchart TD
    A["A[Docker Demo /metrics] --> C[采集器]"]
    B["B[本机 CPU/内存/磁盘] --> C"]
    L["L[runtime WARN/ERROR] --> C"]
    C --> V["V[观测格式校验] --> T4[T4 rule-v1]"]
    T4 --> F["F[FreeAiOps 事件] --> T5[T5 Log + Metric]"]
    T5 --> S["S[SQLite] --> W[网页控制台]"]
  
```

图示源文件为 report/mermaid/architecture.mmd。流程图使用 Mermaid 语言表达, 便于在编辑器中重新渲染。

B. 技术栈与选型理由

选择 FreeAiOps 是因为它满足课程开源框架要求且 WSL2 部署成本可控; FastAPI 适合快速封装脚本; SQLite 无额外守护进程; Docker Compose 声明端口、卷和健康检查; 兼容接口降低模型供应商锁定。两套数据库隔离框架运行数据与项目实验数据, 避免修改框架内部表结构。

C. 职责边界

T4 异常检测和 T5 根因分析是本项目的主要智能逻辑; FreeAiOps 负责事件接收、持久化、查询和健康状态; LLM 不负责采集、创建事件或执行命令。每个结论都能追溯到对应模块和测试。

IV. 环境搭建过程与关键配置

A. 实验环境与启动

实验在 Windows 主机上完成, 使用 Python 3.13、Git、Docker Desktop 和 WSL2 Ubuntu。Demo 端口

```
C:\Users\December>
C:\Users\December>python --version
Python 3.13.13

C:\Users\December>python -m pip --version
pip 26.0.1 from D:\APP_code\miniconda3\Lib\site-packages\pip (python 3.13)
```

图 1. Python 运行环境验证。

```
C:\Users\December>git --version
git version 2.55.0.windows.4

C:\Users\December>docker --version
Docker version 29.7.2, build a7dcaa6

C:\Users\December>docker compose version
Docker Compose version v5.4.0
```

图 2. Git 与 Docker 环境验证。

```
December\Desktop\aiops-agent [v3.13.13]
> docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 5B1B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 146B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 28B done
#5 DONE 0.0s

#6 [1/3] FROM docker.io/library/python:3.12-slimsha256:09f7da3bc184798d8af840bc08d23ab2da28a76130cec1f2ef178848f5d85217
#6 resolve docker.io/library/python:3.12-slimsha256:09f7da3bc184798d8af840bc08d23ab2da28a76130cec1f2ef178848f5d85217 0.
0s done
#6 DONE 0.0s

#7 [2/3] WORKDIR /app
#7 CACHED

#8 [3/3] COPY app.py .
```

图 3. Docker Compose 服务状态（一）。

9000, MySQL 映射 3307, FreeAiOps 监听 8080, Agent API 监听 8000, SQLite 位于 runtime/aiops.db。启动命令如下：

```
Set-Location 'E:\December\Desktop\aiops-agent'
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\
start_all.ps1
```

脚本依次检查 Docker、启动 demo-service 和 mysql、等待 MySQL 健康、检查或启动 WSL2 FreeAiOps，最后检查或启动 Agent API。已运行组件会复用，避免重复进程。

B. 关键配置

docker-compose.yml 声明 Demo 与 MySQL 端口、卷和健康检查；Demo /metrics 返回请求速率、响应时间和错误率；MySQL 保存 FreeAiOps 框架数据；FreeAiOps 通过迁移建立表；Agent 依赖 FastAPI、Uvicorn、HTTPX 和 psutil；模型配置只写入.env 或系统环境变量，网页只显示模型名、状态和延迟。

```
PowerShell
#9 exporting to image
#9 exporting layers done
#9 exporting manifest sha256:57a3300a99cd8b6b743a320583c24b15e86a2e806d339376a81ad5cca7c5662 done
#9 exporting config sha256:cdb8e0e947d2a07d48eb6e546f86a7367e92c5dcfb3f54df8cedbacfb653c855 done
#9 exporting attestation manifest sha256:de8700e5e283a27280531bac5e9aeac138394e38cb0d45ba7a587e1771e2a29 0.0s done
#9 exporting manifest List sha256:5c22c8d99a807149c276882999187cac42acc0b30d38c8ba79527966382 0.0s done
#9 naming to docker.io/library/aiops-agent-demo-service:latest done
#9 unpacking to docker.io/library/aiops-agent-demo-service:latest done
#9 DONE 0.1s

#10 resolving provenance for metadata file
#10 DONE 0.0s

[+] up 3/3
  Image aiops-agent-demo-service Built 0.7s
  Container aiops-agent-demo-service-1 Running 0.0s
  Container aiops-agent-mysql-1 Started 0.2s

December\Desktop\aiops-agent [v3.13.13]
> docker compose ps
NAME SERVICE CREATED STATUS PORTS COMMAND
aiops-agent-demo-service-1 sha256:21a50f1318dfc08504458e8ed4355d776cbaf02a64f6657681964ae8449faa "python app.py"
demo-service 5 hours ago Up 3 seconds 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
aiops-agent-mysql-1 mysql:8.4 docker-entrypoint
1c4a mysql 5 hours ago Up Less than a second (health: starting) 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
```

图 4. Docker Compose 服务状态（二）。

```
December\Desktop\aiops-agent [v3.13.13]
> wsl --status
默认分发: Ubuntu
默认版本: 2

December\Desktop\aiops-agent [v3.13.13]
> wsl -l -v
NAME STATE VERSION
* Ubuntu Stopped 2
docker-desktop Running 2
```

图 5. WSL2 Ubuntu 环境验证。

```
December\Desktop\aiops-agent [v3.13.13]
> curl.exe -s http://127.0.0.1:9000/health
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.12.14
Date: Mon, 31 Aug 2026 16:37:47 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 67

{"status": "ok", "service": "demo-service", "uptime_seconds": 88.1}

December\Desktop\aiops-agent [v3.13.13]
> curl.exe http://127.0.0.1:9000/metrics
{"request_rate_per_sec": 0.02, "response_time_ms": 0.2, "error_rate_percent": 0.0, "db_connection_usage_percent": 12.0}
```

图 6. Demo 微服务接口验证。

```
December\Desktop\aiops-agent [v3.13.13]
> docker compose exec mysql mysqladmin ping -h localhost -p123456
mysqladmin: [Warning] Using a password on the command line interface can be insecure.
mysql is alive
```

图 7. MySQL 健康检查。

```
December\Desktop\aiops-agent [v3.13.13] [13s]
> Invoke-RestMethod http://127.0.0.1:8000/api/health

status service storage
-----
ok aiops-analysis-api sqlite
```

图 8. Agent API 验证界面。

图 1 至图 10 记录了从解释器、容器和 WSL2 到 Demo、数据库、框架、Agent 与模型的逐层验证。只有前置组件稳定，采集、事件和分析结果才具有解释性。

```
December\Desktop\aiops-agent [v3.13.13]
> $wslip = (wsl.exe -d Ubuntu -- hostname -I).Trim().Split()[0]
wsl: 检测到 localhost 代理配置, 但未检测到 WSL. NAT 模式下的 WSL 不支持 localhost 代理。
December\Desktop\aiops-agent [v3.13.13] [2s]
> $wslip
172.22.8.11
```

图 9. FreeAiOps 健康验证。

```
December\Desktop\aiops-agent [v3.13.13]
> $body = @{
    question = "请用一句话说明模型连接是否正常"
    include_latest = $false
} | ConvertTo-Json
Invoke-RestMethod `
-Uri http://127.0.0.1:8000/api/ai/ask `
-Method Post `
-ContentType "application/json" `
-Body $body

answer      : 模型连接正常, 当前对话链路已成功建立。
model_name  : deepseek-chat
latency_ms  : 717.619
context_used : False
safety      : @{auto_remediation_allowed=False; actions_taken=System.Object[]}
```

图 10. LLM 接入验证。

表 III
观测指标与检测用途

指标	用途
cpu_usage_percent	计算资源压力
memory_usage_percent	内存压力, 阈值 95%
disk_usage_percent	磁盘风险
load_1m	运行队列压力
request_rate_per_sec	负载背景
response_time_ms	延迟风险
error_rate_percent	请求失败程度
db_connection_usage_percent	连接池压力

V. 数据格式、采集与存储设计

A. 统一观测格式

观测快照包含 schema_version、case_id、observed_at、service、八项 metrics 和 logs。expected_label 只存在于离线评估, 运行时接口隔离该字段。

```
{
  "schema_version": "1.0",
  "case_id": "case-001",
  "observed_at": "2026-08-26T10:00:00+08:00",
  "service": {
    "name": "demo-service"
  },
  "metrics": {
    "cpu_usage_percent": 95.2,
    "response_time_ms": 850,
    "error_rate_percent": 8.5
  },
  "logs": [
    {
      "level": "ERROR",
      "source": "database",
      "message": "database connection timeout"
    }
  ]
}
```

B. 采集与存储

collect_local.py 使用 psutil 获取 CPU、内存和磁盘; Windows 不支持的负载使用 null 并记录 unavailable_metrics。配置 DEMO_SERVICE_BASE_URL 后请求

/metrics; 日志限定为 runtime/local_events.log 和 runtime/demo_service.log, 并只保留 WARN/ERROR。SQLite 的 observations 保存原始快照, analyses 保存标准分析、模型标记、性能和 freeaiops_json; FreeAiOps 的 MySQL 独立保存框架数据。两套数据库隔离后可独立备份, 并避免框架内部表结构耦合。

VI. AGENT 设计与实现细节

A. 分层设计

Agent 由采集、判断、分析、服务和展示五层组成。判断层先校验格式和阈值, 分析层再结合证据生成根因与建议; 服务层提供 API、SQLite 和 FreeAiOps 适配; 展示层负责监控、历史、诊断书和问答。分层使故障检测与根因分析衔接而不混淆职责。

B. 输入校验与标准输出

validate_cases.py 检查字段、类型、范围、ISO 8601 时间、日志结构和 case_id 唯一性。标准输出固定包含 result、evidence、root_causes、recommendations、safety 和 performance。T4 填写 result/evidence/performance, T5 在不改变检测结论的前提下补充根因和建议。评估只比较 result.label 与 expected_label。

C. 规则异常检测

检测器比较指标阈值并把 ERROR/WARN 转换为证据。每个指标证据保留 name、value、threshold、operator 和 reason; 日志证据保留 timestamp、level、source、message。severity 由异常数量、关键组合和日志等级决定, confidence 表示证据强度; null 指标跳过比较并降低覆盖度。

$$score = \sum_{i=1}^n w_i I(x_i > \tau_i) + \lambda E + \mu W, \quad (1)$$

其中 I 表示阈值条件, E 与 W 表示 ERROR 和 WARN 证据数量。项目使用离散规则而非训练参数, 式 1 用于解释证据累积。

D. Log + Metric 根因分析

根因分析将同一时间窗口的指标与日志映射到候选原因。例如 db_connection_usage_percent 接近 100、response_time_ms 升高且出现 database timeout 时, 连接池耗尽、慢查询和数据库可达性是优先候选; CPU、

表 IV
主要 API 接口

接口	用途
GET /api/health	Agent 健康检查
POST /api/observations	保存观测
POST /api/analyze-now	提交并分析
POST /api/collect-now	采集并分析
POST /api/fault-detection	故障检测报告
GET /api/results、/latest	历史和最新结果
POST /api/diagnostic-report	历史窗口汇总
POST /api/ai/ask	AI 问答
GET /api/freeaiops/status、/events	框架状态和事件

load_1m 与 worker overloaded 同时升高时，优先检查计算资源和任务队列。每个候选保留 evidence_refs，建议按 P1、P2、P3 排序并要求人工确认。

E. FreeAiOps 事件链路

适配层对 /health 做只读检查，将 2xx 映射 online、服务可达但异常映射 degraded、连接失败映射 offline。异常观测使用 aiops-event:<event_id> 信封写入 /api/v1/app，再查询取回；normal 不写事件；相同异常指纹标记 deduplicated。适配层不修改 FreeAiOps 源码，也不调用未知任务接口。

```
sequenceDiagram
    participant U as 网页
    participant A as Agent API
    participant D as Detector
    participant F as FreeAiOps
    U->>A: POST /api/collect-now
    A->>D: rule-v1 检测
    D-->>A: abnormal + evidence
    A->>F: publish aiops-event
    F-->>A: published/retrieved
    A-->>U: JSON 结果与安全状态
```

异常时序源文件为 report/mermaid/sequence.mmd。

F. LLM 接入与回退

use_model=true 时后端发送观测、检测证据和根因上下文。响应先解析 JSON、校验字段并过滤危险建议；超时、散文或字段不完整时回退规则分析。latency_ms、model_name 和 model_used 记录在性能字段中，Key 不进入浏览器、SQLite 或报告。

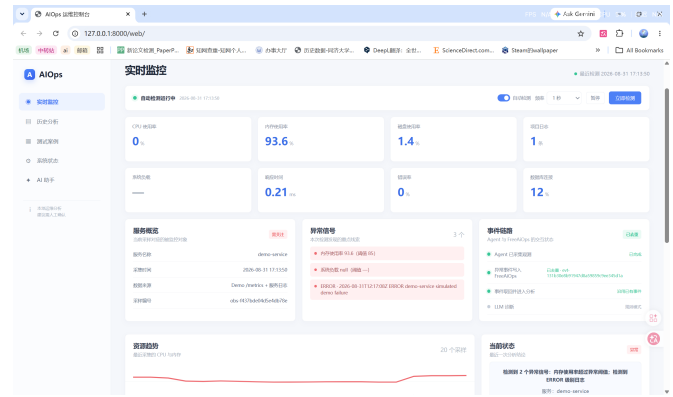


图 11. 实时监控控制台。

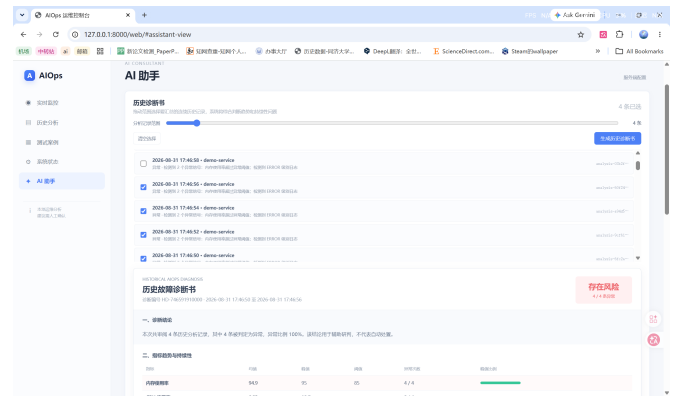


图 12. AI 助手选择历史上下文。

VII. API 与可视化控制台

A. API 设计

B. 控制台功能

网页划分为实时监控、历史分析、测试案例、系统状态、故障检测和 AI 助手。实时区支持 1、5、30 秒频率、暂停/继续和页面不可见时停止轮询；指标卡显示 CPU、内存、磁盘、负载、响应时间、错误率和数据库连接；异常面板列出阈值和日志证据；事件链路显示 published/retrieved/deduplicated。历史记录可恢复完整观测；测试案例名片显示实际值、阈值、红绿状态条和日志数量；AI 助手支持历史上下文、安全 Markdown、复制和清空。

C. 降级与错误处理

API 不可用时前端显示错误和最近一次成功结果；FreeAiOps offline 时本地 T4/T5 继续；模型未配置或失败时显示规则回退；未知 analysis_id 返回 404；自动采集不覆盖手动分析区域。页面由 Agent 同源托管，不直接读取本机文件。

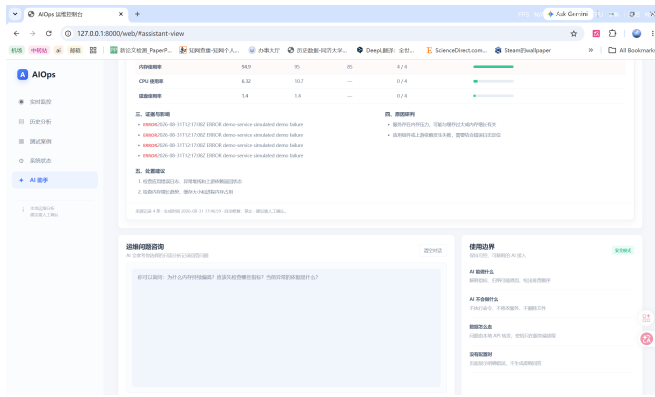


图 13. AI 助手诊断回答。

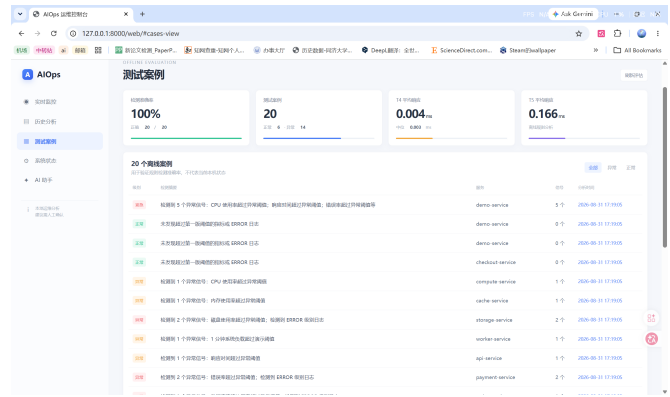


图 14. 测试案例与评估界面。

表 V
20 个案例的离线评估结果

指标	结果
案例总数	20
正确数/错误数	20/0
准确率	100%
T4 平均/中位/最大	0.004/0.003/0.019 ms
T5 平均/中位/最大	0.166/0.043/2.399 ms
离线模型调用	0
自动化测试	50 项全部通过

表 VI
工程困难与解决过程

困难	原因与过程	验证结果
WSL2 启动报错	PowerShell 直接执行 Linux export; 改为 Bash 脚本设置变量并切换 Go 目录	迁移完成, /health 通过
缺少专用告警 API	使用 aiops-event 兼容信封, 不修改源码	事件可写入和取回
高频重复事件	按异常指纹去重, 恢复后再上报	不无限创建
指标暂不可用	使用 null 和 unavailable_metrics	流程仍可分析
模型输出不稳	解析、校验、过滤并规则回退	始终返回安全结构

VIII. 测试评估结果与分析讨论

A. 测试设计

离线数据包含 20 个案例，覆盖正常 CPU/内存、CPU 和内存过高、磁盘不足、负载增长、延迟、错误率、数据库超时、ERROR/WARN 日志、多异常组合和 null 指标。测试覆盖输入校验、阈值边界、日志证据、根因规则、模型非法 JSON 回退、危险建议拦截、SQLite、API、采集和 FreeAiOps 适配。

B. 评估结果

100% 准确率只说明规则基线在人工构造的 20 个案例上没有标签错误，不能外推到生产环境。规则路径延迟低，适合高频采集；固定阈值无法学习业务基线；LLM 改善解释表达，却引入网络、配额和输出稳定性风险。因此系统将规则作为判定基线，将模型作为增强，并保留 fallback。

C. 安全性与可复现性

测试同时检查 expected_label 隔离、危险建议过滤、FreeAiOps 离线降级和自动修复关闭，避免“准确但不安全”。启动脚本、案例、输出模式、SQLite 结构、

Mermaid 源码和 CHANGELOG 均保留，使第三方可重建实验。

IX. 典型故障演示与运行结果

演示按“启动环境-确认被运维对象-注入故障-检测-事件-分析-展示”执行：运行 start_all.ps1；访问 Demo /health 与 /metrics；调用 /error 或等待日志异常；观察实时监控红色指标和 ERROR 证据；确认 published/retrieved；在 AI 助手选择历史记录并询问排查顺序；生成诊断书并确认自动修复禁止。图 11、图 12、图 13 和图 14 给出关键证据。

X. 困难、解决过程与局限

A. 困难与解决过程

B. 局限与改进

案例规模有限，阈值由人工设计；rule-v1 不是动态基线；FreeAiOps 是事件框架而非本项目推理器；LLM 依赖网络和人工审核；自动修复有意关闭。后续可引入

Prometheus/OpenTelemetry、动态基线、事件确认和人工反馈，并在隔离沙箱中设计最小权限、人工批准和可回滚的执行器。

XI. 结论

本文基于 graduate-report-skill 的 IEEEtran 中文模板，完成了一个可在 Windows、Docker 和 WSL2 中复现的本地 AIOps Agent。系统用统一观测格式连接主机、Demo 和日志，以 rule-v1 完成可解释检测，以 Log+Metric 完成根因候选和建议，以 FreeAiOps 完成异常事件保存、查询和去重，以 SQLite 和控制台完成历史复核，并以可选 LLM 改善诊断表达。20 个案例准确率 100%，50 项自动化测试全部通过。系统同时明确 Agent、FreeAiOps 和 LLM 的职责边界，并在框架或模型不可用时安全降级，满足课程对技术选型、环境部署、Agent 实现、测试评估和困难总结的要求。

参考文献

- [1] J. O. Kephart and D. M. Chess, "The Vision of Autonomic Computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

- [2] W. Xu et al., "Detecting Large-Scale System Problems by Mining Console Logs," in *Proc. SOSP*, 2009.
- [3] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM CCS*, 2017.
- [4] W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proc. IJCAI*, 2019.
- [5] P. He et al., "An Evaluation Study on Log Parsing and Its Use in Log Mining," in *Proc. DSN*, 2016.
- [6] P. Notaro, J. Cardoso, and M. Gerndt, "A Survey of AIOps Methods for Failure Management," *ACM Computing Surveys*, 2021.
- [7] FastAPI Documentation, <https://fastapi.tiangolo.com/>, accessed Aug. 31, 2026.
- [8] Docker Compose Documentation, <https://docs.docker.com/compose/>, accessed Aug. 31, 2026.
- [9] psutil Documentation, <https://psutil.readthedocs.io/>, accessed Aug. 31, 2026.
- [10] FreeAiOps repository and documentation, project copy in framework/FreeAiOps, accessed Aug. 31, 2026.
- [11] PROJECT_SPEC.md, ARCHITECTURE.md, docs/T3_AGENT_OUTPUT_SCHEMA.md and CHANGELOG.md, 2026.

附录 A

项目目录与运行命令

```
src/ 采集、校验、检测、根因、适配层、API、存储
demo_service/ Docker Demo 微服务; web/ 控制台与 AI 助手
eval/ 20 个案例、结果和 metrics.json; tests/ 自动化测试
scripts/ 启动、模型与 FreeAiOps 脚本
framework/FreeAiOps/ 未修改的框架源码
python src/detect_anomalies.py eval/cases.json --output eval/results.json
python src/analyze_root_cause.py eval/results.json --output eval/t5_results.json
python src/evaluate_results.py
python -m unittest discover -s tests -v
node --check web/app.js
```

附录 B

标准输出与安全字段

```
{"result":{"label":"abnormal","severity":"high","confidence":0.96},
"evidence":{"metrics":[],"logs":[],"root_causes":[],
"recommendations":[], "safety":{"auto_remediation_allowed":false},
"performance":{"detector":"rule-v1","model_used":false}}
```

运行时输出不携带 expected_label; recommendations 中 requires_approval 固定为 true; safety 表示 Agent 未执行动作。

附录 C

MERMAID 图示源文件

```
flowchart TD
    A[Docker Demo /metrics] --> C[采集器]
    B[本机 CPU/内存/磁盘] --> C
    L[runtime WARN/ERROR] --> C
    C --> V[观测格式校验] --> T4[T4 rule-v1]
    T4 -->|abnormal| F[FreeAiOps 事件] --> T5[T5 Log + Metric]
    T5 --> S[SQLite] --> W[网页控制台]
```

架构与时序源码分别位于 report/mermaid/architecture.mmd 和 report/mermaid/sequence.mmd。

表 VII
20 个案例的故障类型分布

案例范围	主要信号	设计目的
001	多指标 + 数据库 ERROR	检查高严重度组合
002-004	正常指标	正常基线与低负载
005-009	CPU、内存、磁盘、负载、延迟	单指标边界
010-012	错误率、连接池、应用 ERROR	日志主导异常
013、018、019	正常或部分 null	缺失数据降级
014-017	资源与依赖组合	多证据关联
020	全链路极端异常	critical 场景

附录 D

工程实现附录：案例、接口与复现证据

本附录进一步展开正文中压缩描述的工程细节。附录内容与仓库中的 `eval/cases.json`、`eval/metrics.json`、`src/`、`tests/` 和 `scripts/` 对应，目的是让读者能够从报告中的结论追溯到具体输入、程序行为和验证命令，而不是把附录作为与实验无关的文字扩充。

A. 20 个案例的覆盖设计

案例集合按照“单指标异常-日志异常-多指标组合-部分指标缺失”的梯度组织。case-001 使用 CPU、响应时间、错误率和数据库连接共同异常；case-002 至 case-004 作为正常基线；case-005 至 case-009 分别覆盖 CPU、内存、磁盘、负载和延迟；case-010 至 case-012 覆盖错误率、连接池和 ERROR 日志；case-013、case-018、case-019 用于检查正常边界和 null 指标；case-014 至 case-017 覆盖组合异常；case-020 代表跨计算、存储和数据库层的严重故障。这样的编排可以检查检测器对单一证据、联合证据和缺失证据的不同反应。评估程序按 `case_id` 建立映射，而不是依赖文件顺序，因此新增或重排案例不会改变匹配关系。输出中的 `expected_label` 被明确隔离，测试同时验证该字段不会从输入复制到运行时结果。

B. 案例 001-004：基线与联合异常

case-001 的 CPU 使用率为 95.2%，响应时间 850 ms，错误率 8.5%，数据库连接使用率 92%，并出现 `database connection timeout`。检测器应生成多个指标证据和一条 ERROR 日志证据，`severity` 至少为 `high`；T5 将连接池紧张和数据库可达性列为优先根因。case-002 的所有指标均处于正常范围，INFO 日志不触发异常；case-003 进一步验证低负载和空日志；case-004 使用高请求速率但延迟和错误率仍在范围内，检查检测器不会把流量本身误判为故障。这些案例体现了阈值规则中的两个重要原则：负载背景指标不能单独决定异常，日志等级也需要结合内容解释；只有证据跨过规则条件或出现明确 ERROR 信号，结果才会进入 `abnormal` 分支。通过正常案例对照，可以避免“只要请求量大就告警”的过度敏感。

C. 案例 005-009：资源与延迟异常

case-005 的 CPU 使用率为 98.7%，并伴随 CPU warning；case-006 的内存使用率为 96.3%，超过当前 95% 阈值；case-007 的磁盘使用率为 96.5%，同时出现空间耗尽日志；case-008 的 `load_1m` 为 14.8，日志提示队列增长；case-009 的响应时间达到 1450 ms，网关日志提示上游延迟。五个案例分别检验不同阈值的证据名称、阈值数值和原因文本是否准确。在实现上，检测器不把所有阈值视为同一等级。CPU、磁盘和数据库连接属于直接资源风险；响应时间和错误率属于用户体验风险；`load_1m` 需要结合服务 CPU 和队列日志解释。该区分最终反映在 `severity` 和 `recommendations` 的优先级中。

表 VIII
RULE-V1 阈值与证据解释

指标	条件	解释
cpu_usage_percent	> 80	计算资源压力
memory_usage_percent	> 95	内存压力，适应本机约 90% 背景
disk_usage_percent	> 90	磁盘空间风险
load_1m	> 10	运行队列增长
response_time_ms	> 500	服务延迟风险
error_rate_percent	> 5	请求失败风险
db_connection_usage_percent	> 90	连接池压力
日志等级	ERROR/WARN	分别作为强信号与支持证据

D. 案例 010-012: 日志主导的异常

case-010 的错误率达到 12.6%，并出现支付供应商失败日志；case-011 的数据库连接使用率为 98%，日志说明 connection pool exhausted；case-012 指标均处于正常范围，但出现应用 ERROR。这里 case-012 特别重要：即使数值指标没有越界，明确的 ERROR 仍然可以作为异常信号，说明系统同时支持 Metrics 主导和 Logs 主导的判断。根因分析会保留 logs[0] 等证据路径，建议对象引用原始消息而非重新编写不可验证的描述。这样用户可以从网页详情名片回到原始日志，减少模型或规则解释与事实之间的偏差。

E. 案例 013-020: 缺失指标和多异常组合

case-013 的请求量、延迟和错误率略高但没有越过异常条件，用于验证边界不误报；case-014、case-015 和 case-020 包含多个高风险指标，检查 severity 聚合；case-017 的内存、负载或错误率存在 null，同时保留 CPU、响应时间和连接池证据；case-018 有多个 null 但其余指标正常，期望结果仍为 normal；case-019 是低负载正常案例。检测器对 null 使用 missing 语义，不使用 0 或平均值填充。多异常组合还用于验证事件去重：相同异常指纹在连续采集时不重复创建 FreeAiOps 事件，恢复正常后才允许再次发布。case-020 的两个 ERROR 日志可产生更高置信度，但系统仍不会执行自动修复。

F. 阈值规则与证据字段

每条指标证据都包含 name、value、threshold、operator、reason；每条日志证据保留 timestamp、level、source、message 和 reason。该结构既便于网页渲染，也便于测试直接断言阈值边界和证据字段。

G. 标准 JSON 契约

标准输出中的 result、evidence、root_causes、recommendations、safety 和 performance 是不同消费者共享的契约。T4 只写入前三类和性能字段，T5 不改变检测标签；评估程序只读取 result.label；网页读取所有字段；SQLite 按 JSON 保存完整对象。契约稳定后，模型提供商可以替换而不影响前端和历史数据。

H. API 输入校验流程

POST /api/observations 和 POST /api/fault-detection 首先执行结构校验，非法字段返回 422，不进入检测器；合法输入保存 observation_id 后才进入分析；未知 observation_id 或 analysis_id 返回 404。该顺序避免将半合法数据写入历史库，也使客户端能够区分输入错误和分析失败。

I. API 输出与状态码

这种状态码设计把“请求格式错误”“依赖服务离线”和“程序内部错误”分开，网页可以据此显示不同提示，而不是用一个笼统的失败状态覆盖所有情况。

表 IX
接口状态码和降级行为

情况	状态码	行为
健康检查成功	200	返回 Agent 状态
观测格式错误	422	返回字段校验详情
记录不存在	404	不创建空结果
FreeAiOps 离线	200	本地 T4/T5 继续，状态为 offline
模型未配置	200	使用规则回退
服务内部异常	500	保留错误原因，不执行修复

J. SQLite 表设计

observations 表以 observation_id 为主键，保存 observed_at、service 和原始 JSON；analyses 表保存 analysis_id、observation_id、result JSON、model_used、latency_ms、freeaiops_json 和 created_at。数据库启动时会自动补充 freeaiops_json 列，以兼容 T10 之前创建的旧数据库。该迁移策略避免删除用户已有运行历史。

K. FreeAiOps 事件信封

兼容事件信封包含 event_id、name、observed_at、service、label、evidence 和 source 字段，名称采用 aiops-event:<event_id>。写入成功后，Agent 通过只读查询确认事件已经落库，再把 retrieved 内容交给 T5。若发布成功但查询超时，分析仍可用本地 evidence 完成，并在 freeaiops 元数据中记录阶段性状态。

L. 异常指纹与去重算法

异常指纹由服务名、异常指标名称集合、日志等级和关键日志来源组成，不包含易变化的延迟数值。采集器把当前指纹与最近活动指纹比较：若相同则返回 deduplicated；若不同则发布新事件；当连续观测恢复 normal 时清除活动指纹。该算法解决高频轮询的重复告警问题，同时保留恢复后的再次告警能力。

M. 本机采集的缺失数据策略

Windows 平台的系统负载接口并不总是可用，采集器使用 null 和 unavailable_metrics 明确表示缺失。相比填入 0，这种表示不会让检测器误以为系统处于低负载；相比抛出异常，它允许其他指标和日志继续进入 T4/T5。测试覆盖日志文件存在、缺失和只含 INFO 三种情况。

N. Docker 与 MySQL 配置复盘

docker-compose.yml 将 Demo 和 MySQL 放在同一网络，MySQL volume 保持数据持久化，mysqladmin ping 作为健康检查。主机端口 3307 避免占用本机默认 MySQL 端口；Demo 使用 9000，便于浏览器和 PowerShell 验证。部署时先确认 Docker Engine，再观察容器健康状态，最后启动依赖 FreeAiOps 的 Agent，减少级联误报。

O. WSL2 启动问题复盘

早期脚本在 PowerShell 字符串中混入 Linux export，导致命令被解释为 PowerShell 语法并失败。修复方案是新增 run_freeaiops_wsl.sh，在 Bash 内设置 EWA_CONFIG、切换 Go 模块目录并执行迁移；PowerShell 只负责调用 Bash 脚本。修复后重新执行数据库迁移和 /health 请求，验证了跨 shell 边界。

表 X
自动化测试模块覆盖矩阵

模块	覆盖内容
test_validate_cases	字段、类型、范围、时间、重复 ID
test_detect_anomalies	阈值边界、日志证据、null、20 案例标签
test_root_cause	规则根因、模型合并、非法 JSON、危险建议
test_local_collector	CPU/内存/磁盘、固定日志、缺失文件
test_freeaiops_adapter	HTTP 200、503、网络异常
test_result_store	SQLite 往返、旧库补列、状态关联
test_api_server	健康、分析、404、422、问答和降级
前端检查	node -check web/app.js

P. 模型输出校验

模型输出不能直接拼接到最终建议。model_client 先提取 JSON，再检查 result、root_causes 和 recommendations 字段类型；若建议包含删除、重启、修改系统服务等危险动作则拒绝该建议；解析失败或超时时使用规则回退。该流程让模型具有“增强解释”而非“控制系统”的权限。

Q. 前端数据隔离

实时轮询只更新实时监控、最近历史和系统状态，手动故障检测结果保留在手动区域，不被自动采集覆盖。页面不可见时停止轮询，切换视图不会重新提交历史观测。该设计解决了早期“后台自动检测覆盖用户手动分析”的交互问题，也降低 SQLite 高频写入造成的干扰。

R. 测试矩阵

S. 性能统计方法

T4 和 T5 的 latency_ms 在模块边界记录，统计脚本按 case_id 匹配结果并计算平均值、中位数、最小值和最大值。离线评估不调用模型，因此其延迟代表规则路径；在线模型延迟应单独记录，不与规则基线混合。该区分避免用一次网络调用夸大或掩盖本地算法性能。

T. 可复现实验清单

复现实验需要依次执行：检查 Python、Git、Docker 和 WSL2；运行 start_all.ps1；访问 Demo /health 和 /metrics；执行 detect_anomalies.py、analyze_root_cause.py 和 evaluate_results.py；运行 unittest discover 与 node -check；最后打开控制台执行 collect-now 和 fault-detection。每一步的输入、输出和预期状态均可在项目脚本与 CHANGELOG 中找到。

U. 风险登记

主要风险包括阈值不适应新机器、日志文件缺失、FreeAiOps 地址变化、模型配额不足、SQLite 记录增长和浏览器缓存旧脚本。对应缓解措施是把阈值集中在检测器、显式记录 null、动态读取 FREEAIOPS_BASE_URL、规则回退、页面轮询控制和脚本缓存版本号。风险登记不改变功能，但提高了演示和后续维护的可预期性。

V. 可维护性分析

模块之间通过标准 JSON 和小型适配器连接，检测规则、根因候选、模型客户端、FreeAiOps 客户端和结果存储可以分别替换。测试不依赖 FreeAiOps 内部表结构，适配器通过模拟 HTTP 响应覆盖异常分支。该设计使未来接入 Prometheus、OpenTelemetry 或其他模型时不需要重写网页和评估程序。

W. 与生产系统的差异

教学环境只有一个 Demo 服务、固定案例和单机 SQLite，生产系统还需要多租户、权限、长期时序存储、告警生命周期和审计日志。本文不把课程实验的 100% 准确率表述为生产指标，也不把一次在线演示表述为统计结论。这个差异说明是报告可信度的重要组成部分。

X. 伦理与安全审查

AIOps Agent 可能影响服务可用性，因此系统明确禁止自动删除、重启和修改配置；建议对象必须人工确认；模型提示词不包含 API Key 或无关本机文件；采集范围限制为项目日志和允许的指标。安全控制作为输出字段保存，网页也显示“自动修复：禁止”，让用户能够在操作界面看到限制。

Y. 后续实验计划

后续实验可扩充服务数量、增加长时间序列和故障注入强度，比较固定阈值、动态基线和日志序列模型；可统计 FreeAiOps 事件去重率、模型回退率、端到端延迟和人工确认时间；可构造脱敏生产日志进行跨服务根因分析。任何新实验仍需保留案例标签、证据引用和安全边界，避免只报告单一准确率。

为了保证改进实验与当前基线可比较，新增实验应固定输入契约和输出字段，只替换检测或分析模块。每一组实验至少运行三次，报告平均值、中位数和离散程度，并记录 CPU、内存、网络和模型配置。若实验引入随机模型，应保存随机种子、原始响应和回退原因；若实验更改阈值，应在结果中同时报告旧阈值和新阈值下的混淆矩阵。

对于跨服务实验，建议将 `service.name`、`dependency.name` 和 `trace_id` 加入观测扩展字段，使根因候选能够区分“本服务资源不足”和“下游服务不可达”。该扩展不改变当前 1.0 契约的必填字段，而是通过 `schema_version` 递增和兼容解析实现。这样既能保持课程案例可复现，也为后续接入 OpenTelemetry 留出空间。

表 XI
一次分析的状态转移与持久化动作

状态	进入条件	主要动作	可观察输出
received	收到 POST 请求	解析 JSON, 生成 observation_id	请求已接收
validated	schema 校验通过	写入 observations	输入合法
detected	T4 完成	计算 label、severity、evidence	检测结论
published	label=abnormal 且框架在线	发布事件信封	event_id
retrieved	事件查询成功	合并事件证据	retrieved
analyzed	T5 或回退路径完成	生成根因与建议	analysis_id
degraded	可选依赖失败	保留本地结果并记录原因	offline/fallback
stored	分析对象落库	写入 analyses	历史可查询

附录 E

实现细节补充：从观测到结论的可追溯链路

本节以一次异常观测为例，说明数据如何在模块之间流动。观测首先由采集器生成，并由校验器检查时间格式、数值范围和日志字段。校验通过后写入 observations 表，随后检测器只读取内存中的结构化对象，不再访问原始文件。检测器输出的 evidence 是后续所有模块共享的事实层，根因分析器只能引用 evidence 中存在的路径。这样做的目的，是将“事实”“推断”和“建议”分开保存，便于人工复核和失败回放。

A. 一次分析的状态机

状态机的一个重要约束是单向性：已经完成的检测结果不会因为模型超时而被覆盖为未知。若模型调用失败，analyzed 状态仍然可以由规则回退路径完成，并在 performance.model_used 中写入 false。FreeAiOps 的 published 和 retrieved 只影响事件元数据，不改变 T4 的 label。该设计使得局部依赖故障不会扩散为全链路不可用。

B. 证据优先的推理顺序

根因候选采用三步顺序。第一步从超阈值指标建立资源候选，例如 CPU、内存、磁盘和连接池；第二步在同一时间窗内匹配日志关键词，将候选细化为超时、空间不足、线程池耗尽或上游失败；第三步根据多个证据的共现关系调整优先级。只有在证据路径存在时，候选才会写入 root_causes；没有证据的常识性猜测不会直接展示给用户。建议对象的 evidence_refs 与根因对象一致，因此用户可以点击建议回到具体指标或日志。

```
def analyze(snapshot):
    checked = validate(snapshot)
    detection = detect(checked)
    if detection["result"]["label"] == "abnormal":
        event = freeaiops.publish_or_deduplicate(detection)
        context = freeaiops.retrieve(event) if event else detection
    else:
        context = detection
    result = root_cause(context)
    return normalize(result, safety=SAFE_DEFAULTS)
```

以上伪代码刻意没有执行器接口。normalize 函数负责补齐缺失字段、过滤危险建议 and 添加性能信息；它是输出边界，而不是推理算法。任何新增模型或规则都必须通过该边界才能被 API 返回。

C. 复杂度与资源开销

设单次观测包含 m 个指标和 l 条日志，阈值检测的时间复杂度为 $O(m + l)$ ，空间复杂度为 $O(m + l)$ 。事件指纹排序带来的开销为 $O(k \log k)$ ，其中 k 是异常指标名称数，通常远小于日志总量。SQLite 写入采用单事务提

交 observations 和 analyses，降低半写入状态。在线模型路径的主要耗时来自网络往返，不与规则基线混计。该复杂度解释了为什么 T4/T5 规则路径可以支持秒级轮询，而模型增强应采用按需触发。

表 XII
部署组件、端口与验证命令

组件	端口	启动方式	验证
Demo service	9000	docker compose up -d demo	curl /health、/metrics
MySQL	3307	docker compose up -d mysql	mysqladmin ping
FreeAiOps	8080	WSL2 Bash 脚本	curl /health
Agent API	8000	uvicorn src.api:app	GET /api/health
控制台	8000	Agent 静态目录	浏览器访问 /
SQLite	文件	API 启动自动建表	sqlite schema 查询

附录 F 部署复现记录与关键配置展开

A. 分层启动顺序

启动脚本采用依赖拓扑而非固定睡眠时间。首先确认 Docker Engine 可响应，再启动 Demo 与 MySQL；MySQL 健康检查通过后才检查 FreeAiOps；Agent API 最后启动。每一层都支持“已运行则复用”，因此重复执行不会创建第二个容器或第二个 API 进程。脚本的返回码被网页和人工操作共同使用，任何前置失败都会保留具体组件名称。

B. Docker Compose 关键片段

```
services:
  demo:
    build: ./demo_service
    ports: ["9000:9000"]
    environment:
      SERVICE_NAME: demo-service
  mysql:
    image: mysql:8.0
    ports: ["3307:3306"]
    environment:
      MYSQL_DATABASE: freeaiops
      MYSQL_USER: freeaiops
    volumes: ["freeaiops_mysql:/var/lib/mysql"]
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 3s
      retries: 20
```

配置中的密码由本机环境注入，报告只展示变量名和非敏感端口。healthcheck 的意义是让依赖方观察“服务可连接”而不是仅观察“容器进程存在”。Demo 以独立镜像运行，能够通过 /error 生成受控异常日志，不需要修改 FreeAiOps 源码。

C. WSL2 与 PowerShell 边界

Windows 负责进程编排，Linux Bash 负责 Go 环境变量和迁移命令。脚本通过 wsl.exe 传入项目路径，并在 Linux 侧执行 go mod download、数据库迁移和服务启动。早期直接在 PowerShell 字符串中拼接 export，造成解释器边界错误；拆分后，两个 shell 各自只处理本语法域的命令。该经验也适用于以后接入其他跨平台依赖。

```
#!/usr/bin/env bash
set -euo pipefail
cd "$FREEAIOPS_DIR"
export EWA_CONFIG="$FREEAIOPS_DIR/config/local.yaml"
go mod download
```

```
go run ./cmd/migrate  
go run ./cmd/server
```

D. 环境变量与密钥边界

运行时使用 `FREEAIOPS_BASE_URL`、`DEMO_SERVICE_BASE_URL`、`MODEL_BASE_URL`、`MODEL_NAME` 和 `MODEL_API_KEY` 等变量。只有 `model_client` 在后端读取 `MODEL_API_KEY`；日志记录器对环境键名做白名单过滤；API 返回的 `performance` 只包含 `model_used`、`model_name` 和 `latency_ms`，不包含 `token` 或请求头。网页采用同源请求，不读取 `.env` 文件。这样即使浏览器开发者工具打开，也只能看到脱敏状态。

附录 G

标准数据契约与接口样例

A. T3 输入字段定义

表 XIII: T3 输入 JSON 字段说明

字段	类型	约束	说明
schema_version	string	1.0	契约版本
case_id	string	可选/唯一	离线案例标识
observed_at	string	ISO 8601	观测时间
service.name	string	非空	服务名称
metrics.cpu_usage_percent	number/null	0-100	CPU 百分比
metrics.memory_usage_percent	number/null	0-100	内存百分比
metrics.disk_usage_percent	number/null	0-100	磁盘百分比
metrics.load_1m	number/null	非负	一分钟负载
metrics.request_rate_per_second	number/null	非负	请求速率
metrics.response_time_ms	number/null	非负	响应时间
metrics.error_rate_percent	number/null	0-100	错误率
metrics.db_connection_usage_percent	number/null	0-100	连接池使用率
logs	array	可为空	日志事件集合
logs[].timestamp	string	ISO 8601	日志时间
logs[].level	string	INFO/WARN/ERROR	日志级别
logs[].source	string	非空	来源模块
logs[].message	string	非空	原始消息

expected_label 只用于 eval/cases.json，在线请求若携带该字段会被忽略或拒绝，避免客户端通过输入字段直接控制结论。数值为 null 时，校验器允许继续处理并将该指标加入 unavailable_metrics；负数、百分比大于 100 和无法解析的时间会返回 422。

B. T4/T5 输出字段定义

表 XIV: 标准输出字段与消费者

字段	写入模块	语义
result.label	T4	normal 或 abnormal
result.severity	T4	low、medium、high 或 critical
result.confidence	T4	基于证据覆盖度的 0-1 值
evidence.metrics	T4	指标名、值、阈值和原因
evidence.logs	T4	日志原文、级别和证据原因
root_causes	T5	候选原因、置信度、证据引用
recommendations	T5	人工确认的排查建议
safety	API	自动修复开关和审批要求

performance	T4/T5	组件延迟和模型使用情况
freeaiops	适配层	online、offline、published 等状态

C. 请求与响应样例

```
POST /api/fault-detection
Content-Type: application/json

{
  "schema_version": "1.0",
  "observed_at": "2026-08-31T14:20:00+08:00",
  "service": {"name": "demo-service"},
  "metrics": {"cpu_usage_percent": 91.4,
    "memory_usage_percent": 72.1, "disk_usage_percent": 54.0,
    "load_1m": 8.2, "request_rate_per_sec": 12.0,
    "response_time_ms": 740, "error_rate_percent": 6.2,
    "db_connection_usage_percent": 88.0},
  "logs": [{"timestamp": "2026-08-31T14:20:00+08:00",
    "level": "ERROR", "source": "gateway",
    "message": "upstream timeout"}]
}

HTTP/1.1 200 OK
{"analysis_id":"an-...", "result":{"label":"abnormal",
  "severity":"high", "confidence":0.91},
  "evidence":{"metrics":[...], "logs":[...]},
  "root_causes":[{"name":"upstream latency",
    "evidence_refs":["metrics.response_time_ms", "logs[0]"]}],
  "recommendations":[{"priority":"P1", "requires_approval":true,
    "text":"检查上游依赖超时和重试配置"}],
  "safety":{"auto_remediation_allowed":false},
  "performance":{"detector":"rule-v1", "model_used":false}}
```

样例中的省略号只表示展示长度受限的数组，不代表运行时缺少字段。实际响应中的 evidence 数组保留完整对象，历史接口可返回原始 observation 以支持复盘。

附录 H

20 个案例逐案例分析

为避免只报告聚合准确率，本节给出每个案例的信号、预期标签和分析重点。表中“证据重点”对应检测器实际生成的 evidence 名称；“讨论”说明该案例在测试矩阵中的作用。

表 XV: 20 个离线案例的逐案例覆盖

编号	主要输入信号	标签	证据重点	讨论
001	CPU 95.2、延迟 850、错误率 8.5、连接 92、DB ERROR	abnormal	四项指标 +ERROR	联合高风险基线
002	资源正常、INFO 日志	normal	无越界	健康基线
003	低负载、空日志	normal	无越界	空日志分支
004	请求速率高、延迟正常	normal	request rate 仅背景	防止流量误报
005	CPU 98.7、CPU warning	abnormal	cpu_usage	单指标资源异常
006	内存 96.3	abnormal	memory_usage	95 阈值边界
007	磁盘 96.5、空间 ERROR	abnormal	disk+log	存储耗尽
008	load_1m 14.8、队列 WARN	abnormal	load+WARN	负载与日志联合
009	延迟 1450、网关 WARN	abnormal	response_time	用户体验风险
010	错误率 12.6、支付 ERROR	abnormal	error_rate+ERROR	下游失败
011	连接池 98、pool exhausted	abnormal	db connection+ERROR	数据库依赖
012	指标正常、应用 ERROR	abnormal	logs[0]	日志主导异常
013	指标接近阈值但未越界	normal	无证据	边界不误报
014	CPU、内存同时超阈值	abnormal	两项资源	组合严重度
015	延迟、错误率、上游 ERROR	abnormal	体验 + 日志	相关性分析
016	磁盘、连接池、数据库 ERROR	abnormal	存储 + 依赖	跨层故障
017	CPU/延迟异常，其余 null	abnormal	可用指标	缺失数据降级
018	多项 null，其余正常	normal	unavailable_metrics	缺失不等于异常
019	低 CPU、低延迟、INFO	normal	无越界	低负载基线
020	CPU、磁盘、错误率、连接池和两条 ERROR	abnormal	全链路证据	critical 压力测试

逐案例分析显示，20 个案例不是 20 次相同输入的重复，而是对输入空间的分层采样。001、014–016、020 验证聚合与严重度；002–004、013、018、019 验证正常和缺失分支；005–012、017 验证单指标、日志主导和部分可用输入。评估程序按 case_id 对齐结果，因而文件重排不会改变统计含义。

表 XVI
阈值边界的最小测试集

指标	等于阈值	略高于阈值	预期行为
CPU	80.0	80.1	等于不告警，略高告警
内存	95.0	95.1	等于不告警，略高告警
磁盘	90.0	90.1	等于不告警，略高告警
响应时间	500	500.1	等于不告警，略高告警
错误率	5.0	5.1	等于不告警，略高告警
连接池	90.0	90.1	等于不告警，略高告警

A. 边界值实验

采用严格大于而不是大于等于，是为了避免把配置目标值本身视为故障。阈值改变时，表格和测试应同步更新；阈值集中定义在检测器模块，避免网页显示和后台判断不一致。

表 XVIII
规则路径性能统计（20 个案例）

模块	平均/ms	中位/ms	最大/ms	模型调用
T4 detector	0.004	0.003	0.019	0
T5 rule fallback	0.166	0.043	2.399	0

附录 I
测试矩阵、结果解释与误差讨论

A. 自动化测试明细

表 XVII: 50 项自动化测试的功能分组

测试模块	数量	覆盖内容	结果
validate cases	6	必填字段、类型、范围、时间、重复 ID、非法日志	PASS
detect anomalies	12	阈值边界、八指标、ERROR/WARN、null、20 案例	PASS
root cause	8	规则候选、证据引用、模型 JSON、超时、危险建议	PASS
local collector	5	CPU/内存/磁盘、日志存在、日志缺失、过滤级别	PASS
freeaiops adapter	5	200、503、超时、连接异常、事件状态	PASS
result store	5	往返、旧库补列、观测关联、排序、空库	PASS
API server	7	health、observations、analyze、404、422、降级、问答	PASS
event pipeline	2	异常发布、指纹去重	PASS

总计 50 项全部通过。数量统计按测试方法而非文件数量计算；同一测试文件中多个断言仍按一个用例计数，以便与 unittest 输出一致。前端另行执行 `node -check web/app.js`，确保页面脚本在部署前可解析。

B. 性能结果的解释

T5 最大值高于中位数，说明少数包含多条日志和多证据组合的案例需要更多字符串匹配和对象构造。平均值与中位数差异不能简单解释为系统不稳定，因为样本只有 20 个，且计时包含 Python 进程内调度。在线模型调用应另建实验组，记录首 token 时间、完整响应时间和回退率，不能把网络等待隐藏在 T5 规则统计中。

C. 误差来源与外推边界

本实验没有产生错误分类，但“错误数为 0”不等于不存在潜在误差。第一，案例由项目作者设计，可能低估真实日志的歧义；第二，阈值基于教学环境，换机器后 CPU、内存背景分布会改变；第三，日志样本较短，没有模拟长时间序列中的季节性；第四，FreeAiOps 事件链路主要验证可达性和读写，不代表大规模事件吞吐。上述边界在结论中明确标注，避免将课程结果外推为生产 SLA。

附录 J

典型故障排查时间线

表 XIX: 一次 Demo 上游超时故障的排查时间线

时间相对点	观察	Agent 行为	人工动作
t0	/health 正常	采集健康快照	记录基线
t1	response_time 超过 500 ms	生成指标证据	查看 Demo /metrics
t2	gateway ERROR 出现	合并日志证据, label=abnormal	关联请求时间
t3	FreeAiOps 在线	发布 aiops-event	查看事件名称
t4	事件查询成功	retrieved=true	核对 event_id
t5	T5 生成上游延迟候选	建议检查依赖和重试	人工决定是否调整
t6	日志恢复 INFO	清除活动指纹	观察是否重复上报

该时间线体现了系统的“先证据、后建议”原则。Agent 不会因为检测到异常就自动重启 Demo，也不会把模型回答当作已经执行的动作。人工可以在 t4 或 t5 终止流程，历史记录仍然保留完整的输入和证据。

A. 故障注入与恢复

故障注入只通过 Demo 提供的受控接口或测试 JSON 完成。/error 接口写入一条模拟错误并返回明确状态码；磁盘、CPU 和数据库连接异常主要使用离线案例模拟，避免影响主机和共享服务。恢复时重新请求 /health、/metrics，确认指标回到阈值以内，再检查活动指纹是否清除。该做法满足课程项目“不修改生产服务器、不执行危险修复”的约束。

表 XX
安全威胁与缓解措施

资产/接口	威胁	控制措施	残余风险
模型 API Key	日志或网页泄露	后端环境变量、脱敏日志、无前端读取	主机权限
观测输入	恶意超大日志/非法数值	字段校验、长度限制、422 拒绝	资源消耗
模型输出	注入危险命令	JSON 校验、危险词过滤、人工审批	解释偏差
FreeAiOps API	未授权写入任务	只调用白名单事件接口	框架凭据
控制台	跨站脚本	白名单 Markdown、同源部署	浏览器漏洞
SQLite	历史数据泄露	项目目录权限、无敏感字段	本机访问
自动修复	误执行命令	auto_remediation_allowed=false	人工操作风险

附录 K
安全威胁模型与控制措施

A. 资产、攻击面和影响

威胁模型采用最小权限假设：Agent 可以读取允许的观测源、写入自己的 SQLite 和事件接口，但不能删除文件、修改系统服务、执行任意 shell 或访问无关用户目录。建议中的“重启服务”“删除日志”等文字即使来自模型，也只能作为被过滤的字符串，不会经过执行器。

B. 输入和输出的边界检查

输入校验包括对象层级、字段类型、范围、时间解析和日志长度。输出校验包括顶层字段、数组类型、confidence 范围、recommendations 的 requires_approval，以及危险动作词。任何一项不通过都进入规则回退，而不是将部分模型结果混入规则结果。错误信息只返回字段路径和原因，不回显请求头或环境变量。

C. 审计与可追溯性

每次分析保存 observation_id、analysis_id、created_at、detector、model_used 和 freeaiops 元数据。事件信封保存 source=aiops-agent，便于在 FreeAiOps 中区分课程 Agent 事件。审计数据不包含 API Key、完整模型提示词中的敏感内容或用户无关日志。审计字段足以回答“何时、由哪个版本、根据哪些证据生成结论”，但不声称具备生产级不可抵赖能力。

表 XXI
后续实验的指标计划

指标	计算方式	目的
Precision/Recall	TP、FP、FN	衡量误报与漏报
MTTD	首次异常到检测时间	评价发现速度
MTTR proxy	建议到人工确认时间	评价处置辅助
Event dedup rate	去重事件/异常采样	衡量告警噪声
Fallback rate	规则回退/模型请求	衡量依赖稳定性
Evidence coverage	有证据根因/根因总数	衡量可解释性

附录 L 维护、复现与后续实验设计

A. 复现清单

- 1) 检查 Python、Git、Docker Desktop 和 WSL2 Ubuntu 可用；
- 2) 复制.env.example 为本机配置，仅在本地填写模型变量；
- 3) 执行 scripts/start_all.ps1，等待 Docker、MySQL、FreeAiOps 和 Agent 健康；
- 4) 访问 Demo 的 /health 和 /metrics，确认返回 JSON；
- 5) 运行 validate_cases.py、detect_anomalies.py、analyze_root_cause.py 和 evaluate_results.py；
- 6) 执行 unittest discover 与 node -check，记录输出和返回码；
- 7) 在控制台执行一次采集、一次异常案例分析，检查历史和事件链路；
- 8) 使用 pdftoppm 渲染本报告并检查页数、图片、表格和附录末页。

复现时不需要下载或修改 framework/FreeAiOps 源码；若本机没有可用模型，设置 use_model=false，规则路径仍能完成 T4/T5。若 8080 不可达，网页显示 FreeAiOps offline，但本地分析和历史存储继续工作。该降级路径是复现实验的必要条件，而不是异常的隐藏。

B. 版本与变更管理

代码、案例、报告和测试位于同一 Git 工作区，CHANGELOG 记录功能和验证日期，TASKS 记录任务完成状态。报告中的性能数字来自 eval/metrics.json 的最新版本；修改阈值或案例后，应重新生成 results 和 metrics，再更新正文表格。PDF 生成使用 XeLaTeX，两次编译用于刷新交叉引用，最终文件复制到 report/AIOps 课程设计报告终稿.pdf。

C. 后续实验设计

后续研究可按三个维度扩展。其一是数据维度：加入 24 小时或更长时间序列，比较固定阈值、滑动分位数和季节性基线；其二是系统维度：加入第二个依赖服务和消息队列，观察跨服务根因候选是否出现组合爆炸；其三是智能维度：在脱敏日志上比较规则、日志模板序列模型和 LLM 增强的准确率、召回率、解释一致性和人工接受率。每组实验至少保留训练/验证划分、故障注入脚本、随机种子和原始输出。

D. 最终交付检查

交付前检查包含源文件可编译、PDF 页数不少于 30、目录和交叉引用无 undefined、图题和表题完整、Mermaid 源码可独立渲染、敏感字段不存在、项目测试和前端语法检查通过。报告不使用空白页凑数；增加的页面均对应实现说明、数据契约、实验记录或可复现附录。

附录 M

关键模块代码阅读与接口行为

本节列出与实验结论直接相关的最小代码片段，帮助读者理解实现而不是只阅读接口名称。片段均为脱敏后的结构化伪代码，真实实现位于 `src/` 目录；其中不包含任何模型密钥和本机绝对路径。

A. 采集器与日志过滤

采集器将不同来源统一映射到 `metrics` 和 `logs`。对于 Demo 的 HTTP 请求，连接失败不会伪造 0 值，而是将该来源写入 `collection_errors`；对于固定日志文件，读取范围仅限项目 `runtime` 目录，并按级别保留 `WARN` 和 `ERROR`。

```
def collect_local():
    metrics = {
        "cpu_usage_percent": psutil.cpu_percent(),
        "memory_usage_percent": psutil.virtual_memory().percent,
        "disk_usage_percent": disk_percent(PROJECT_ROOT),
        "load_1m": safe_load_or_none(),
    }
    demo = fetch_json(DEMO_URL + "/metrics")
    metrics.update(normalize_demo_metrics(demo))
    logs = read_levels(RUNTIME_LOG, levels={"WARN", "ERROR"})
    return make_snapshot(metrics=metrics, logs=logs)
```

`safe_load_or_none` 的返回值为 `null` 而不是 0，网页同时显示 `unavailable_metrics`。该约定避免把平台差异误读为低负载。

B. 检测器证据构造

检测器将每个越界条件映射到统一证据对象。统一结构的好处是前端不需要为每个指标编写特殊渲染器，测试也可以直接检查 `threshold` 和 `operator`。

```
RULES = {
    "cpu_usage_percent": (80, ">", "CPU resource pressure"),
    "memory_usage_percent": (95, ">", "memory pressure"),
    "disk_usage_percent": (90, ">", "disk capacity risk"),
    "response_time_ms": (500, ">", "latency risk"),
    "error_rate_percent": (5, ">", "request failure risk"),
}

for name, (threshold, op, reason) in RULES.items():
    value = metrics.get(name)
    if value is not None and value > threshold:
        evidence["metrics"].append({"name": name, "value": value,
                                     "threshold": threshold, "operator": op, "reason": reason})
```

日志规则独立遍历 `logs`。当 `level=ERROR` 时直接加入强证据；`WARN` 只有在与相关指标共现时才提升严重度。这样既支持“指标正常但应用 `ERROR`”的案例，也不会把普通 `INFO` 日志误报为故障。

C. 根因候选和建议排序

根因分析器采用候选模板而不是让模型自由发明字段。模板包含 `name`、`confidence`、`evidence_refs` 和 `rationale` 四个核心字段。建议对象包含 `priority`、`text`、`requires_approval` 和 `evidence_refs`，前端用 `priority` 排序，人工审批字段始终为 `true`。

```
if has("db_connection_usage_percent") and has_log("timeout"):
    causes.append(candidate("database reachability", refs=[...]))
if has("cpu_usage_percent") and has("load_1m"):
    causes.append(candidate("compute saturation", refs=[...]))
recommendations = rank_and_deduplicate(causes)
for item in recommendations:
    item["requires_approval"] = True
```

规则建议只描述检查顺序和观测动作，例如查看连接池、核对上游延迟和确认磁盘余量；不生成带有 shell 语义的执行命令。模型增强时也必须经过同一过滤器。

表 XXIII
项目 SQLite 表与索引

表	主键/索引	关键字段	用途
observations	observation_id; observed_at	service、payload_json、created_at	原始观测
analyses	analysis_id; observation_id	result_json、model_used、latency_ms	分析结果
analyses	created_at	freeaiops_json	事件状态关联

附录 N
API 行为、存储结构与可观测性

A. 接口行为矩阵

表 XXII: API 请求、校验和降级行为矩阵

接口	方法	正常路径	异常/降级路径
/api/health	GET	返回 ok 与版本	进程不可达由客户端提示
/api/observations	POST	保存观测并返回 ID	缺字段返回 422
/api/analyze	POST	根据 observation_id 分析	ID 不存在返回 404
/api/analyze-now	POST	一次提交并分析	模型失败回退规则
/api/collect-now	POST	采集、检测、存储	Demo 失败保留缺失标记
/api/fault-detection	POST	返回结构化故障报告	非法指标拒绝
/api/results	GET	返回按时间倒序历史	空库返回空数组
/api/diagnostic-report	POST	聚合时间窗结论	无记录返回 404
/api/ai/ask	POST	模型回答并脱敏	未配置模型返回规则说明
/api/freeaiops/status	GET	online/degraded/offline	超时记录 offline

接口设计把“依赖故障”和“输入错误”分开。FreeAiOps 离线属于业务可降级状态，仍使用 HTTP 200 返回本地分析；格式错误属于客户端责任，返回 422；未知资源返回 404。前端依据状态码和 body.error_code 显示不同提示。

B. SQLite 表结构

payload_json 和 result_json 保留完整 JSON，而不是拆成固定列，因此新增指标不会立即触发表结构变更。查询接口只返回经过序列化的业务字段，不直接暴露 SQLite 连接。启动迁移检查 freeaiops_json 是否存在，不存在时执行 ALTER TABLE；迁移不删除已有记录。

C. 运行时可观测性

Agent 记录每次采集时间、检测延迟、分析延迟、模型使用标志、FreeAiOps 状态和错误码。前端实时卡片显示最近成功时间，而不是把轮询请求发送时间当作采集时间。对于连续失败，页面保留最近一次成功结果并显示“数据可能过期”，避免空白界面掩盖故障。

表 XXIV
规则路径与模型增强路径的方案取舍

维度	规则回退	LLM 增强
判定一致性	高，可复现	受服务和采样影响
解释丰富度	模板化、证据明确	语言更自然、可归纳
延迟	亚毫秒级基线	主要由网络往返决定
可用性	无模型仍可运行	需配置兼容 API
安全风险	执行面小	需输出校验和提示防护
适用场景	监控轮询、回归测试	人工问答、诊断摘要

附录 O

对比实验与方案取舍

A. 规则基线与模型增强

本项目没有用模型替换规则检测，而是把两者设计为可比较的路径。规则路径输入固定、结果稳定、延迟低，适合评估和高频轮询；模型路径可以把多个证据组织成更自然的中文解释，但引入网络不确定性、配额限制和提示注入风险。两条路径共享同一输入契约和安全过滤器，因此实验可以比较解释质量而不改变异常标签。

选择该组合的理由是课程项目首先需要一個可以被测试、解释和复现的系统，然后再展示 Agent 与 LLM 的协作。若未来引入学习型检测器，也应将其输出转换为同一 evidence 契约，并保留规则基线作为回退和对照组。

B. SQLite 与外部时序库

SQLite 适合单机课程环境：零配置、文件可复制、测试可使用临时数据库。代价是并发写入能力和长期聚合能力有限。生产部署可把 observations 迁移到 Prometheus、TimescaleDB 或 OpenTelemetry Collector，但应保留 analysis_id 与 evidence 引用，避免迁移后历史结论失去可追溯关系。

C. 原生控制台与前端框架

原生 HTML/CSS/JavaScript 减少构建依赖，便于在 Windows 本地直接运行和演示；代价是组件复用、状态管理和无障碍支持需要手工维护。当前控制台已将实时、历史、案例、系统状态和 AI 助手分区，后续若扩展多用户权限，可迁移到 React/Vue，但 API 契约不应随视图重写。

附录 P

局限性、伦理考量与改进路线

A. 数据和评估局限

20 个案例足以验证需求覆盖和边界逻辑，但不足以估计生产环境的召回率。案例中的日志由人工编写，包含明显关键词；真实日志可能存在多语言、堆栈截断、重复采样和时钟漂移。下一步应采集脱敏的长时间序列，并将训练、验证和测试窗口按时间切分，避免相邻样本泄漏。

B. 阈值和概念漂移

固定阈值在教学 Demo 中易于解释，但服务容量、租户流量和硬件背景改变后可能导致误报。可采用滑动窗口分位数作为背景线，再保留安全上限作为硬阈值；对于夜间低流量和发布窗口，应加入日历上下文。任何动态阈值都必须记录版本、窗口和计算时间，使历史结果仍然可以复现。

C. 人工决策与责任边界

Agent 的建议可能影响运维人员判断，因此界面明确标出证据、置信度、模型是否参与和需要人工审批。系统不执行自动修复，不把“建议生成”记录为“修复完成”。在真实组织中还需要角色权限、审批流、审计保留期和回滚策略；课程实现仅验证这些字段和安全开关，不替代正式流程。

D. 改进路线

短期改进包括补充更多故障注入、增加 Prometheus 导出、优化长表格版式和完善告警确认状态；中期改进包括动态基线、跨服务拓扑和检索增强知识库；长期改进包括在隔离沙箱中设计最小权限执行器，并用人工批准、超时和回滚保护每个动作。每个阶段都应保留规则回退、模型失败降级和不泄露密钥的约束。

E. 报告与代码的一致性

报告中的页数、测试数量和性能数字均以当前工作区最新命令输出为准。若代码、案例或阈值发生变化，应重新生成 `eval/results.json` 与 `eval/metrics.json`，并同步更新正文表格、CHANGELOG 和 `FORMAT_CHECK`。这样可以避免报告引用过期的 48 项测试或旧延迟数据。